



School of Computer Science

COMP 8347 “Internet Applications and Distributed
Systems”

Summer 2026

Week 4

Database Integration, Templates, Forms, Sessions, Cookies

DR. OLENA SYROTKINA



University of Windsor

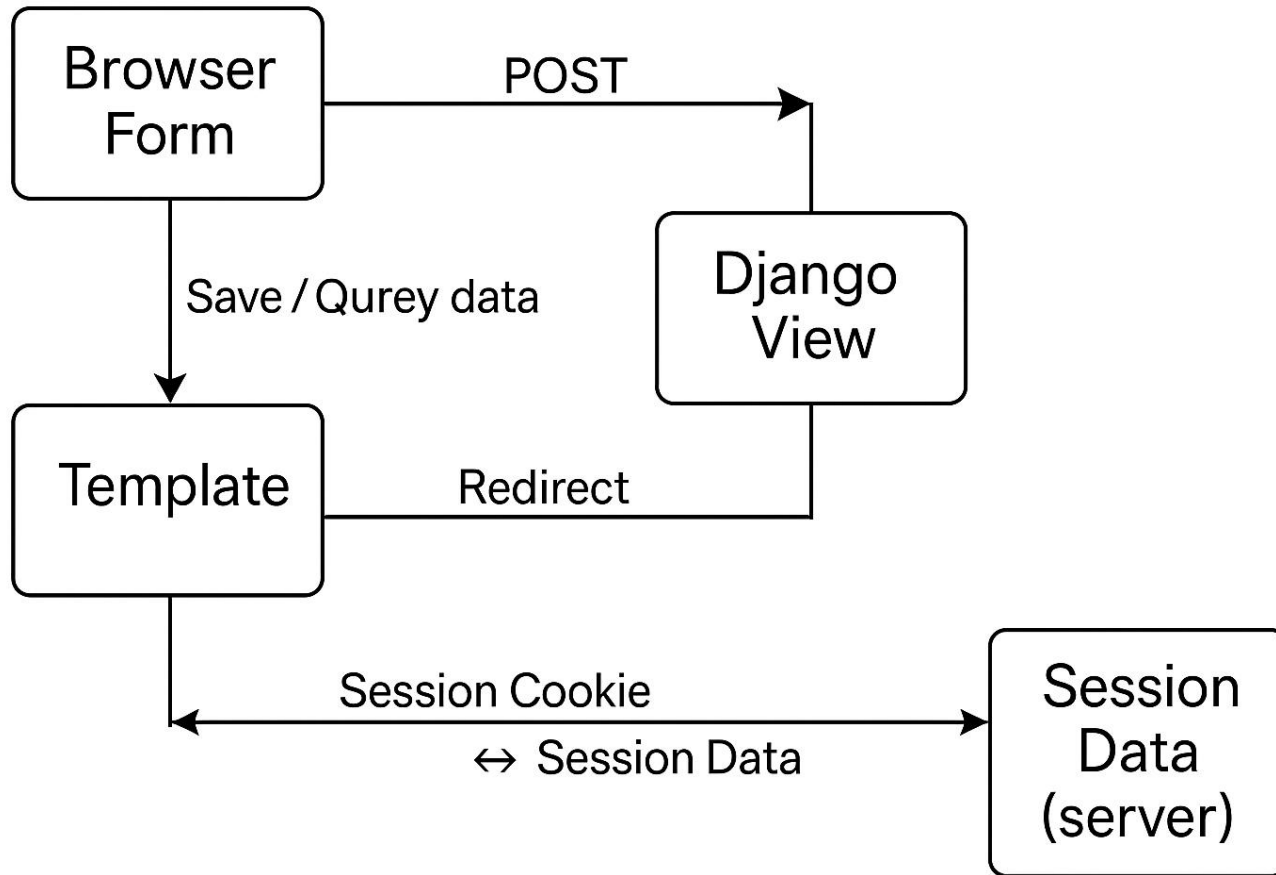
Copyright Notice

This content is protected and may not be shared, uploaded, or distributed. Any unauthorized distribution, reproduction or sharing of this material, including uploading to **CourseHero**, **Chegg**, **StuDocu** or other websites, is **strictly prohibited** and may be subject to legal action. Students are granted access to this material solely for their personal use and may not use it for any other purpose without the express written consent of the instructor. Thank you for respecting our intellectual property rights.

Agenda

- Linking models to templates (dynamic queries, template filters)
- Django Forms – creating and validating user input
- Saving form data to the database
- Sessions & Cookies – tracking visits and preferences
- Group Debugging Challenge

Introduction



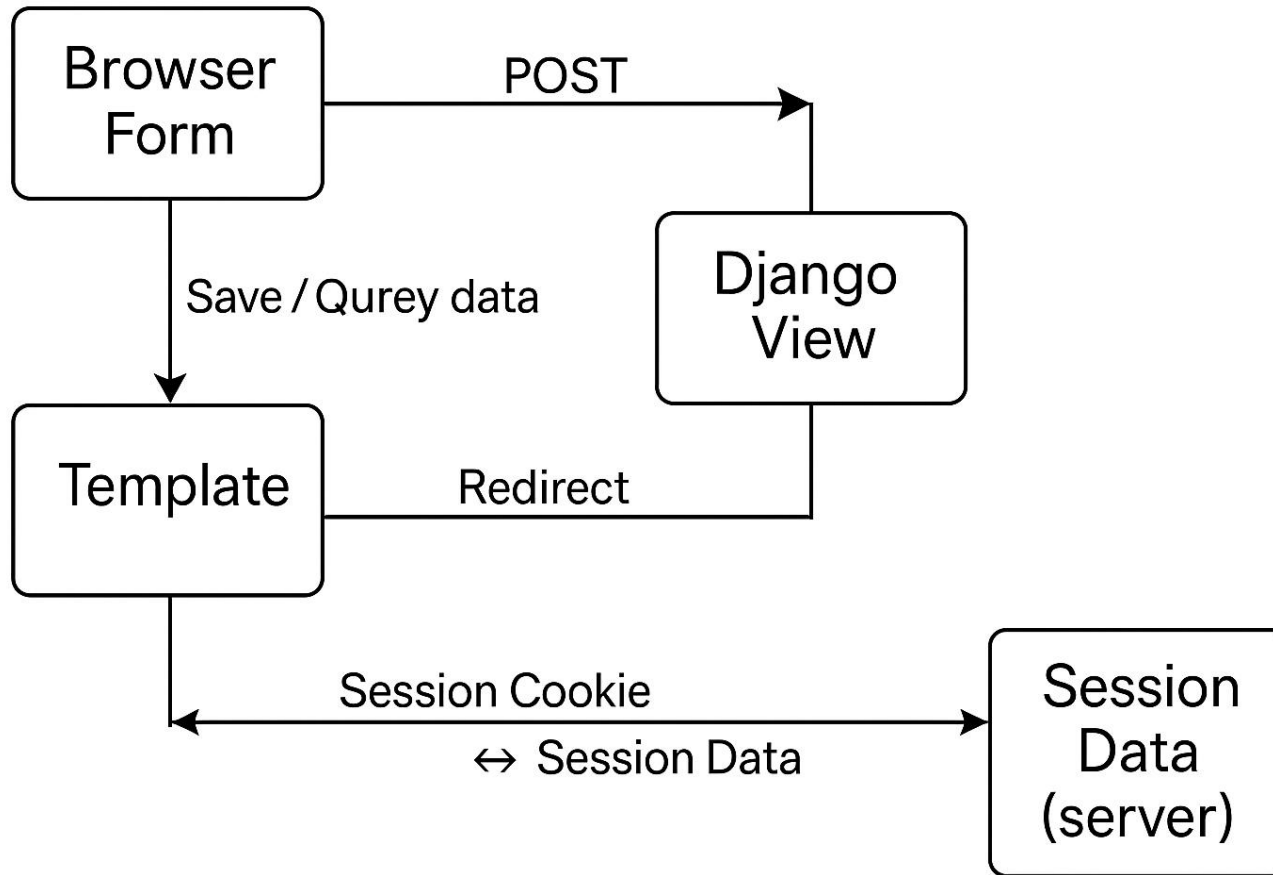
1. Browser Form → Django View (POST)

- A user fills out a form in their browser (for example, login, registration, or data entry).
- When they click Submit, the browser sends the data to the server using an HTTP POST request.
- This request is handled by a Django View function or class.

Browser Form → POST → Django View → DB → Redirect → Template

Session Cookie ↔ Session Data (server)

Introduction



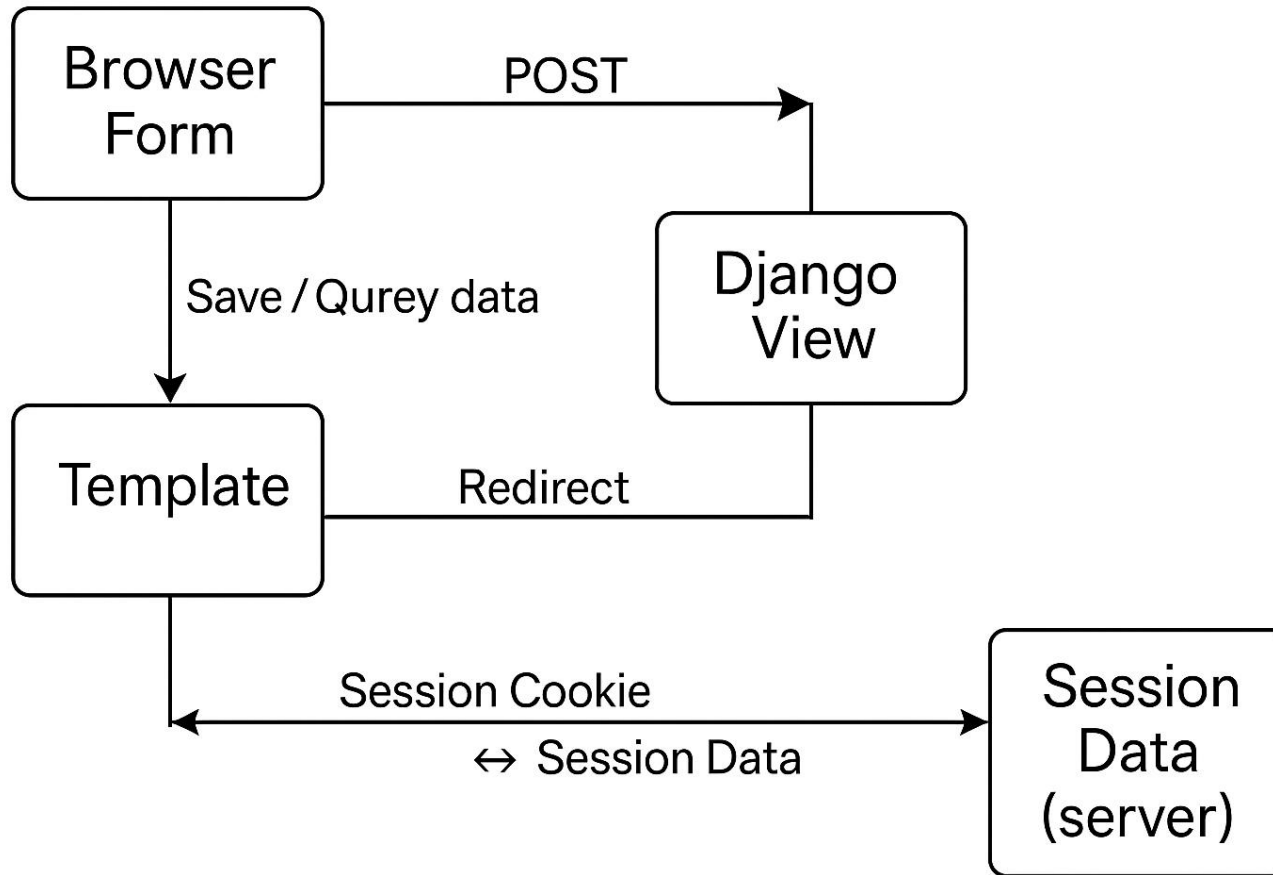
Browser Form → POST → Django View → DB → Redirect → Template

Session Cookie ↔ Session Data (server)

2. Django View → Template (Redirect / Save Data)

- The Django View processes the request:
 - ✓ It can save data to the database.
 - ✓ Or it can query data (fetch results).
- After processing, the view usually redirects the user to another page or renders a template (HTML page).
- Redirect is used to prevent duplicate form submissions (Post/Redirect/Get pattern).

Introduction



Browser Form → POST → Django View → DB → Redirect → Template

Session Cookie ↔ Session Data (server)

3. Template → Browser

- The template generates the HTML page that is sent back to the user's browser.
- The browser displays the updated page (e.g., “Your form has been submitted successfully”).

4. Session Data (Server ↔ Browser)

- Django uses sessions to remember data between requests (e.g., logged-in user, shopping cart).
- The server stores session data (in a database, cache, or file).
- The browser only stores a session cookie (a unique session ID).
- Each time the browser makes a request, it sends the session cookie → Django matches it with the session data stored on the server.

Linking Models to Templates

Example: Show students who joined today

Open `classroom/views.py` and add the following code:

```
from django.utils import timezone
from django.db.models import Count

def dashboard(request):
    today = timezone.now().date()
    stats = {
        "total": Student.objects.count(),
        "joined_today": Student.objects.filter(join_date=today).count(),
        "by_day": Student.objects.extra({'day':
"date(join_date)"}).values('day').annotate(c=Count('id')),
    }
    return render(request, "classroom/dashboard.html", {"stats": stats})
```

Linking Models to Templates

Example: Show students who joined today

Open `classroom/templates/classroom/dashboard.html` and add the following code:

```
{% extends "classroom/base.html" %}

{% block content %}

<div class="page">

    <h1>Dashboard</h1>

    <p>Total students: {{ stats.total }}</p>

    <p>Joined today: {{ stats.joined_today }}</p>
```

```
<ul>

    {% for d in stats.by_day %}

        <li>{{ d.day }} - {{ d.c }}</li>

    {% endfor %}

</ul>

</div>

{% endblock %}
```

Django Forms (User Input)

Example: Create a ModelForm

Open `classroom/forms.py` and add the following code:

```
from django import forms

from .models import Student

class StudentForm(forms.ModelForm):

    class Meta:

        model = Student

        fields = ["name", "email"]
```

Django Forms (Saving Form Data)

Example: Add a View

Open `classroom/views.py` and add the following code:

```
def add_student(request):  
  
    if request.method == "POST":  
  
        form = StudentForm(request.POST)  
  
        if form.is_valid():  
  
            form.save()  
  
            # track in session (see later)  
  
            request.session['added_count'] = request.session.get('added_count', 0) + 1  
  
            return redirect("student-list")  
  
    else:  
  
        form = StudentForm()  
  
    return render(request, "classroom/add_student.html", {"form": form})
```

Django Forms (Saving Form Data)

Now open **config/urls.py** and add the following code:

```
from classroom.views import add_student

urlpatterns += [path("students/add/", add_student, name="add-student")]
```

And now we will create a template **add_student.html**

```
{% extends "classroom/base.html" %}

{% block content %}

<div class="page">

    <h1>Add a Student</h1>

    <form method="post">
```

```
        {% csrf_token %}

        {{ form.as_p }}

        <button type="submit">Save</button>

    </form>

</div>

{% endblock %}
```



What is CSRF?

How Django Protects Against CSRF

- Django includes **CSRF middleware**.
- It requires every **POST form** in your templates to include a **unique token**:

```
{% csrf_token %}
```

This token is:

- ✓ Generated by Django.
- ✓ Stored in your session.
- ✓ Sent with your form.

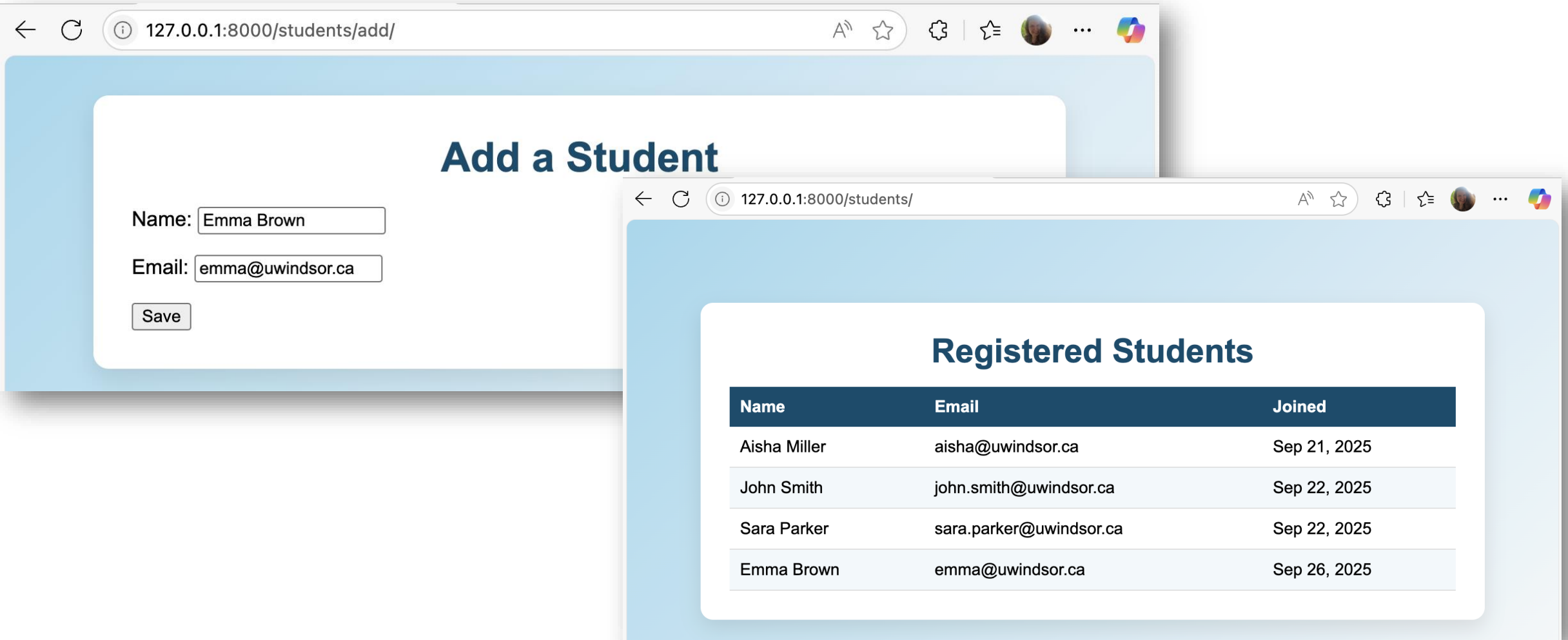
When the form is submitted:

- ✓ Django checks if the token matches the one in the session.
- ✓ If it's missing or invalid → Django rejects the request.

- ✓ This ensures only **genuine forms** from your site can be submitted.

Django Forms

Let's run the application: `python manage.py runserver` and open `http://127.0.0.1:8000/students/add/`



The first screenshot shows a web browser at `127.0.0.1:8000/students/add/` displaying a form titled "Add a Student". The form has two input fields: "Name:" with the value "Emma Brown" and "Email:" with the value "emma@uwindsor.ca". Below the fields is a "Save" button.

The second screenshot shows a web browser at `127.0.0.1:8000/students/` displaying a table titled "Registered Students". The table has three columns: "Name", "Email", and "Joined". The table contains four rows of student data.

Name	Email	Joined
Aisha Miller	aisha@uwindsor.ca	Sep 21, 2025
John Smith	john.smith@uwindsor.ca	Sep 22, 2025
Sara Parker	sara.parker@uwindsor.ca	Sep 22, 2025
Emma Brown	emma@uwindsor.ca	Sep 26, 2025

Sessions

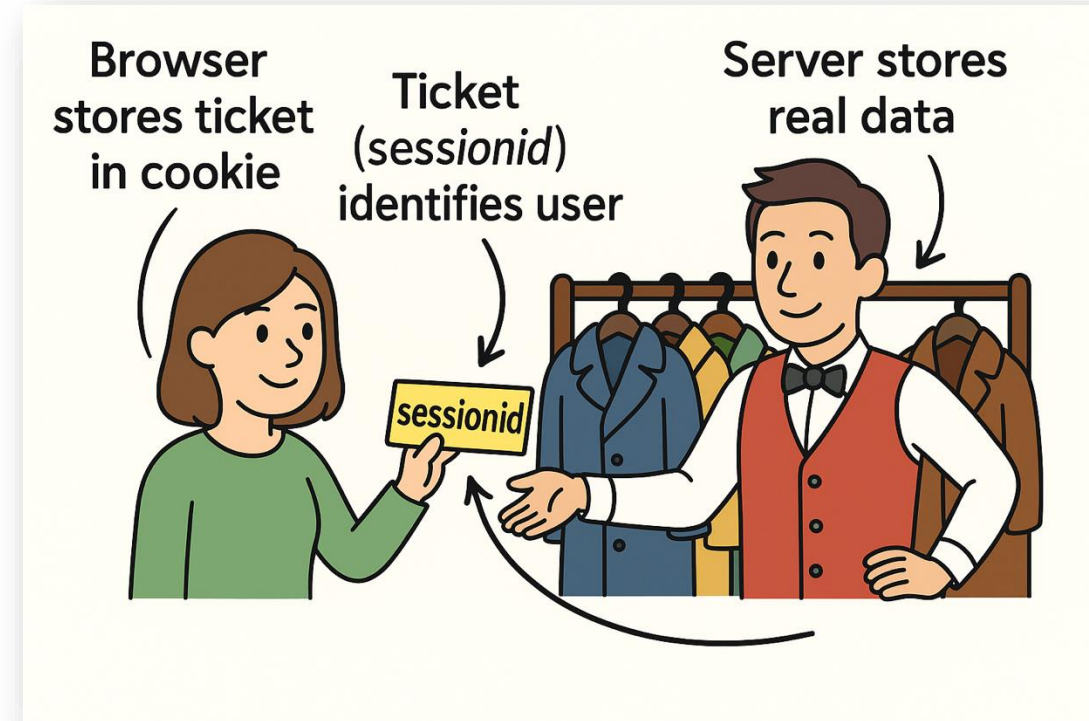
What are Sessions?

A **session** is a way for a web application (like Django) to remember information about a specific user **across multiple requests**.

- Normally, **HTTP is stateless**: every request is independent.
 - ✓ Example: If you log in on one page, the next page wouldn't "remember" you unless we store that info.
- **Sessions fix this problem**: they give each user a unique ID and let the server store data tied to that ID.

How do Sessions Work?

- User makes a request → Django creates a session and gives the user a session ID.
- Browser stores the session ID in a cookie (usually called sessionid).
- Server stores the actual data (e.g. "user is logged in as John").
- Every time the user makes another request, the browser automatically sends the sessionid cookie.
- Django looks up the session data on the server → recognizes the user → continues the experience.



Sessions

A **session** is server-side storage tied to a cookie (**sessionid**) in the browser.

Let's add a View

Open `classroom/views.py` and add the following code:

```
def visit_counter(request):  
    visits = request.session.get("visits", 0) + 1  
    request.session["visits"] = visits  
    return render(request, "classroom/visit_counter.html", {"visits":  
visits})
```

Sessions

Open `classroom/templates/classroom/visit_counter.html` and add the following code:

```
{% extends "classroom/base.html" %}

{% block content %}

<div class="page">

    <h1>Session Demo</h1>

    <p>You have visited this page {{ visits }} times.</p>

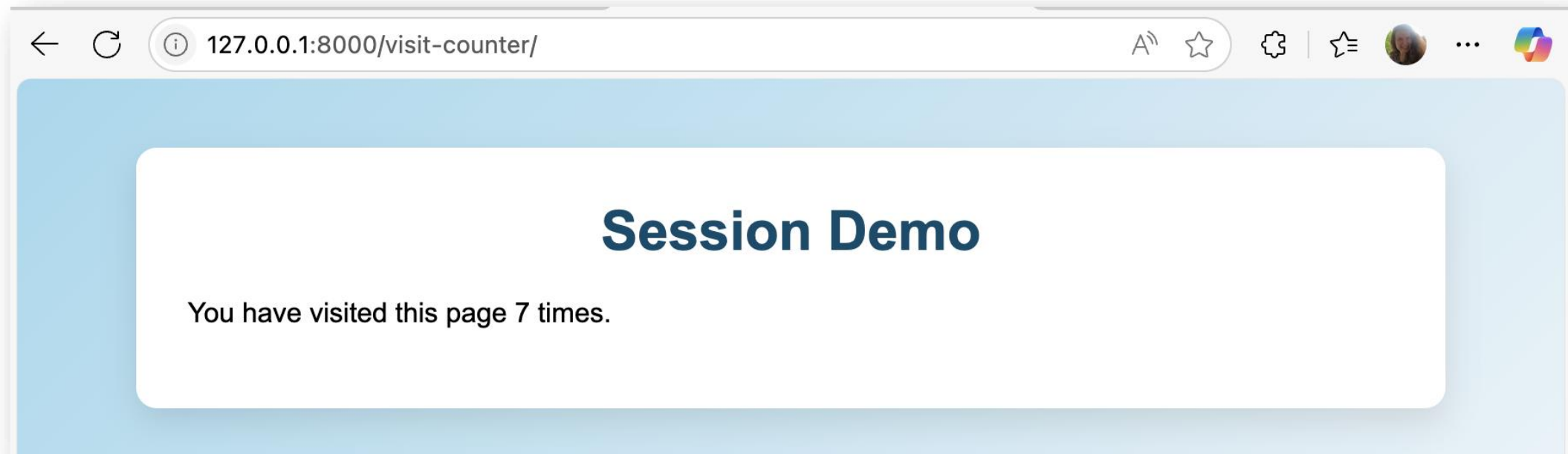
</div>

{% endblock %}
```

Sessions

Now open **config/urls.py** and add the following code:

```
from classroom.views import visit_counter  
  
urlpatterns += [path("visit-counter/", visit_counter, name="visit-counter")]
```



Cookies

What are Cookies?

A cookie is a small piece of data stored in the user's browser.

Each cookie has:

- A name (e.g., sessionid)
- A value (e.g., ajd9281nsa...)
- Optional settings (expiration date, domain, path, secure flag).

How do Cookies Work?

1. Server sets a cookie

- When you log into a Django site, the server sends a response with a Set-Cookie header:

```
Set-Cookie: sessionid=ajd9281nsa...; Path=/; HttpOnly
```

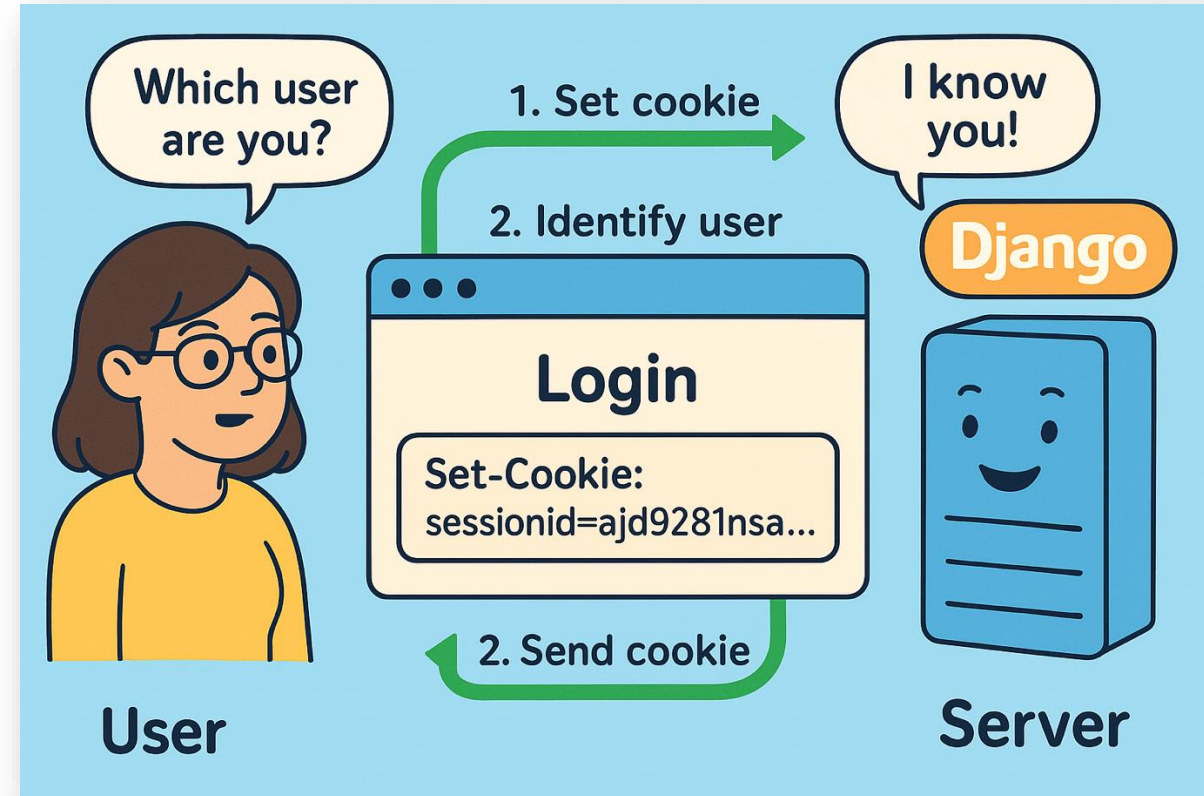
- The browser stores this cookie.
- Browser sends cookie back

2. On the next request, the browser automatically sends the cookie:

```
Cookie: sessionid=ajd9281nsa...
```

3. Server identifies user

Django looks at the cookie value (sessionid), finds the matching session data on the server, and knows **which user you are**.



Cookies

Custom Cookie Example. Go to the views.py file inside your app (e.g., classroom/views.py).

```
from django.http import HttpResponseRedirect

def set_fav_color(request):

    resp = HttpResponseRedirect("Favorite color set!")

    resp.set_cookie("favorite_color", "blue", max_age=3600)

    return resp
```

Cookies

Next Step: Add a URL

In your `urls.py` (e.g., `classroom/urls.py`), add a route for this view:

```
from django.urls import path
from classroom.views import set_fav_color

urlpatterns += [
    path("set-color/", set_fav_color, name="set-fav-color"),
]
```

Now if you go to <http://127.0.0.1:8000/set-color/>
(Make sure your server is running: **`python manage.py runserver`**)

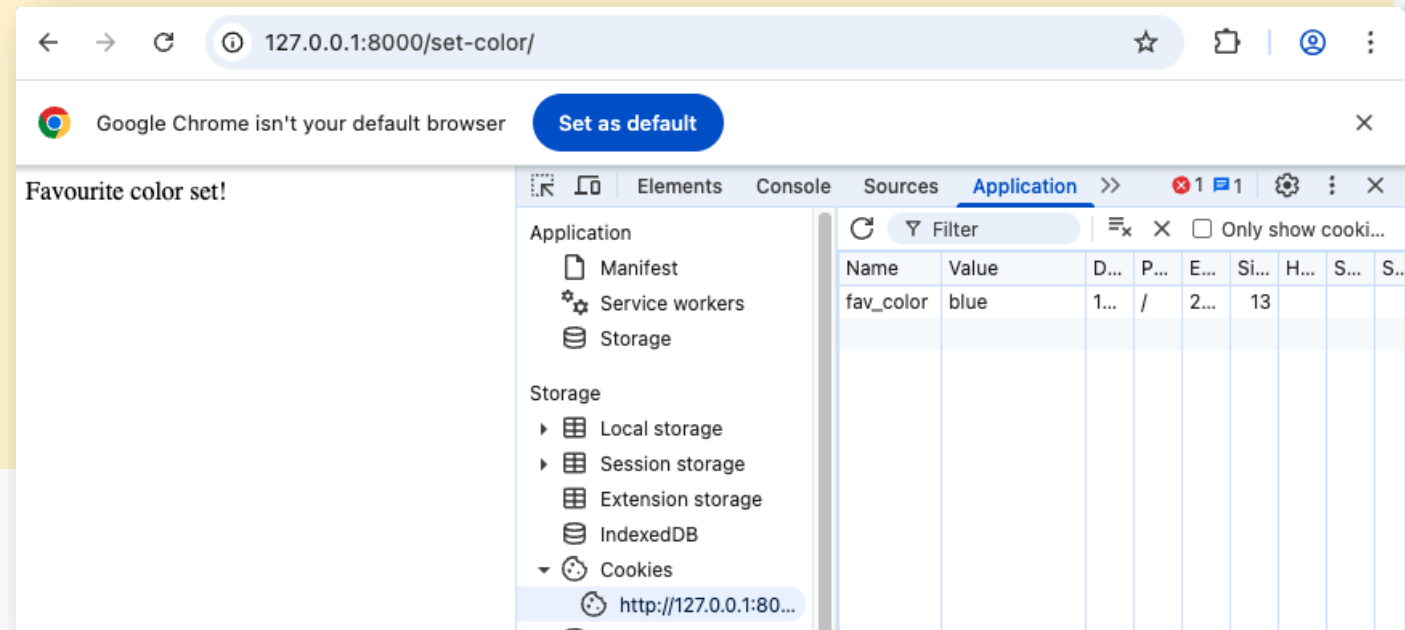
Django will:

- ✓ Return a response Favorite color set!
- ✓ Also set a cookie called **favorite_color=blue** in your browser (lasting 3600 seconds = 1 hour).

Inspecting Cookies in Browser DevTools (set-color)

In Google Chrome (similar in Edge):

1. Open your browser → <http://127.0.0.1:8000/set-color/>.
2. Open DevTools (right-click → Inspect → Application → Storage → Cookies) or (or press Ctrl+Shift+I / Cmd+Opt+I on Mac).
3. Go to the Application tab.
4. In the sidebar, expand Storage → Cookies.
5. Click your site (<http://127.0.0.1:8000>).
6. You should see a cookie!

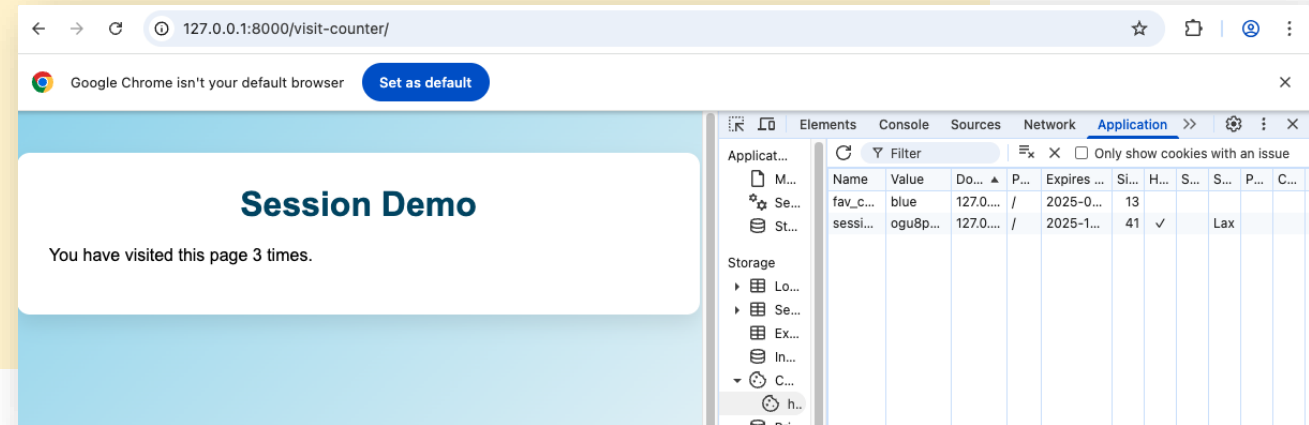


Inspecting Cookies in Browser DevTools (visit-counter)

In Google Chrome (similar in Edge):

1. Open your website (e.g., <http://127.0.0.1:8000/visit-counter/>).
2. Right-click anywhere on the page → Inspect (or press Ctrl+Shift+I / Cmd+Opt+I on Mac).
3. Go to the Application tab.
4. In the sidebar, expand Storage → Cookies.
5. Click your site (<http://127.0.0.1:8000>).
6. You'll see a table of cookies:

- ✓ Name → e.g., sessionid
- ✓ Value → a long random string (unique session key)
- ✓ Domain/Path → scope of the cookie
- ✓ Expires/Max-Age → when it will expire
- ✓ HttpOnly / Secure / SameSite → security settings



Group Debugging Challenge – Guestbook

Fix a broken Guestbook app so users can:

1. Add their name + message.
2. See messages listed with time and author.
3. Use a cookie to remember their name.
4. Track how many entries they've added this session.

What You Have:

1. A Django app called guestbook with:
2. Models: Entry (display_name, message, created_at).
3. Forms: EntryForm (broken).
4. Views: entry_list, add_entry, set_name_cookie, clear_name_cookie (broken).
5. Templates: entry_list.html, add_entry.html (broken).

 **You successfully completed the challenge if:**

- You can add a message without errors.
- Messages display on /guestbook/.
- The name cookie pre-fills the “Display name” field.
- The session counter updates.